

# CS571: Programming Languages

## Lecture Note 2: Encodings

### 1 Introduction

Even though all values in the  $\lambda$ -calculus are functions, it would be nice to somehow have objects which could be worked with like integers and boolean values, and control-flow constructs. In this note, we will build such constructs from the core  $\lambda$ -calculus.

### 2 Encoding Booleans

We first encode Boolean values (TRUE, FALSE) and functions on them (such as AND, IF), such that the expected behavior holds, including statements such as

- $\text{AND TRUE FALSE} = \text{FALSE}$
- $\text{IF TRUE } t \text{ } f = t$ , where  $t$  and  $f$  are arbitrary  $\lambda$ -terms

Notice that here we use the *prefix* form ( $\text{AND TRUE FALSE}$ ), rather than the *infix* form ( $\text{TRUE AND FALSE}$ ) to emphasize that the operation AND is a function with two parameters. Moreover, if-statement in a functional programming language is a function with three parameters: a branch condition; values returned when the branch condition evaluates to TRUE and FALSE respectively.

One way to encode the Boolean values is:

- $\text{TRUE} \triangleq \lambda x \lambda y. x$
- $\text{FALSE} \triangleq \lambda x \lambda y. y$

To encode function IF, we notice that it should be of the form:

$$\text{IF} = \lambda b \lambda t \lambda f. \text{if } b = \text{TRUE} \text{ then } t \text{ else } f$$

Now, the definitions of Boolean values become handy, because  $\text{TRUE } t \text{ } f$  evaluates to  $t$ , and  $\text{FALSE } t \text{ } f$  evaluates to  $f$ . Hence, we only need the term  $b \text{ } t \text{ } f$  in the body of IF:

$$\text{IF} \triangleq \lambda b \lambda t \lambda f. b \text{ } t \text{ } f$$

With the encoding of IF, we can easily define other Boolean operations. For example, we can encode the operation AND:

$$\text{AND} = \lambda b_1 \lambda b_2. \text{IF } b_1 (\text{IF } b_2 \text{ TRUE FALSE}) \text{ FALSE}$$

With  $\beta$ -reduction, that is equivalent to:

$$\text{AND} = \lambda b_1 \lambda b_2. b_1 (b_2 \text{ TRUE FALSE}) \text{ FALSE}$$

Now we check that such encodings work as expected by one example.

$$\begin{aligned} \text{AND TRUE FALSE} &= (\lambda b_1 \lambda b_2. b_1 (b_2 \text{ TRUE FALSE}) \text{ FALSE}) \text{ TRUE FALSE} \quad (\text{by definition}) \\ &= \text{TRUE (FALSE TRUE FALSE) FALSE} \quad (\beta\text{-reduction}) \\ &= (\lambda x \lambda y. x) (\text{FALSE TRUE FALSE}) \text{ FALSE} \quad (\text{by definition}) \\ &= \text{FALSE TRUE FALSE} \quad (\beta\text{-reduction}) \\ &= (\lambda x \lambda y. y) \text{ TRUE FALSE} \quad (\text{by definition}) \\ &= \text{FALSE} \quad (\beta\text{-reduction}) \end{aligned}$$

### 3 Encoding integers

To encode numbers, we'll use Church numerals. Here, the number  $n$  is represented as a higher-order function  $\underline{n}$  which, given another function, returns the  $n$ -fold composition of that other function:  $\underline{n}(f) \mapsto f^n$ . The function  $\underline{n}$  (we use an underline to distinguish a Church numeral from its counterpart in mathematics) encodes number  $n$ . In other words,

$$\underline{n} \triangleq \lambda f z. f^n z = \lambda f z. \underbrace{f(f(\dots f(z)))}_{n \text{ times}}$$

For example,

$$\begin{aligned}\underline{0} &\triangleq \lambda f z. z \\ \underline{1} &\triangleq \lambda f z. f z \\ \underline{2} &\triangleq \lambda f z. f (f z)\end{aligned}$$

For Church numerals, we can define a successor function **SUCC** which takes one Church number and returns its successor. To see how the encoding works, we observe that by definition, **SUCC** should take some Church number  $\underline{n}$  as input, and return the encoding of  $n + 1$ , which is in the form of  $\lambda f z. t$  for some  $t$  yet to be defined. So the goal is write down some  $t$  such that  $t = f^{n+1} z$  given input  $\underline{n}$ .

To define the missing piece  $t$ , we notice that for any  $\underline{n}$ ,  $\underline{n} f z = f^n z$ . So given an input  $\underline{n}$ , we can apply  $f$  to  $\underline{n} f z$  to get  $f^{n+1} z$  (i.e.,  $f (\underline{n} f z) = f^{n+1} z$ ). Hence, we simply define  $t$  as  $f (n f z)$  where  $n$  is the input. In summary, we have the following definition:

$$\text{SUCC} \triangleq \lambda n. \lambda f z. f (n f z)$$

We can also define simple arithmetic, such as **PLUS**. One way of doing that is:

$$\text{PLUS} \triangleq \lambda n_1 n_2. (n_1 \text{ SUCC } n_2)$$

Why this encoding works? Intuitively, a Church number  $\underline{n}$  is a function that takes another function  $f$  and a term  $t$  as inputs, and returns the result of applying  $f$  to  $t$  for  $n$  times. Therefore, we can compute  $\underline{n}_1 + \underline{n}_2$  via  $\underline{n}_1 \text{ SUCC } \underline{n}_2$  (i.e., increment  $\underline{n}_2$  by one for  $n_1$  times).

Following this idea, we can easily encode multiplication **MULT** as:

$$\text{MULT} \triangleq \lambda n_1 n_2. (n_1 (\text{PLUS } n_2) \underline{0})$$

Finally, we check that  $\text{PLUS } \underline{1} \underline{2} = \underline{3}$  based on the definition above.

$$\begin{aligned}\text{PLUS } \underline{1} \underline{2} &= (\lambda n_1 n_2. (n_1 \text{ SUCC } n_2)) \underline{1} \underline{2} \quad (\text{by definition}) \\ &= \underline{1} \text{ SUCC } \underline{2} \quad (\beta\text{-reduction}) \\ &= (\lambda f z. f z) \text{ SUCC } \underline{2} \quad (\text{by definition}) \\ &= \text{SUCC } \underline{2} \quad (\beta\text{-reduction}) \\ &= (\lambda n. \lambda f z. f (n f z)) \underline{2} \quad (\text{by definition}) \\ &= \lambda f z. f (\underline{2} f z) \quad (\beta\text{-reduction}) \\ &= \lambda f z. f ((\lambda u v. u (u v)) f z) \quad (\text{by definition and } \alpha\text{-reduction}) \\ &= \lambda f z. f (f (f z)) \quad (\beta\text{-reduction}) \\ &= \underline{3} \quad (\text{by definition})\end{aligned}$$